

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 4

Objektorientiertes PHP

4.1 Modularisierung

Modularisierung ist ein bewährtes Konzept in der Softwareentwicklung. Teile des Programms, die wiederverwendet werden können oder sollen, werden in eigene Teile ausgelagert. Dies kann eine Funktion sein, die ein kleines Stück Code enthält und mehrfach aufgerufen wird, eine ganze Gruppe von Funktionen und/oder Variablen oder auch ganze Klassen (siehe Abschnitt 4.2). Diese lassen sich in PHP in eigene Dateien auslagern und bequem in verschiedenen anderen Dateien wieder inkludieren. Dies verbessert nicht nur die Struktur, sondern erhöht auch die Übersicht, da sich z. B. globale Konfigurationsvariablen schneller finden lassen.

4.1.1 Dateien inkludieren

PHP bietet vier Sprachkonstrukte an, um den Inhalt von externen Dateien in einer anderen Datei zu inkludieren:

- `include`,
- `require`,
- `include_once` und
- `require_once`.

Die Inhalte der eingebundenen Dateien werden wie PHP-Dateien angesehen, d. h. PHP-Code muss (wie in anderen PHP-Dateien auch) in den PHP-Klammern (`<?php ?>`) stehen. Sind in den inkludierten Dateien Variablen oder Funktionen vorhanden, so ist deren Gültigkeitsbereich derselbe, als ob sie direkt an der Datei an dieser Stelle stünden. Variablen, die vor einer inkludierten Datei existierten, sind in dieser dann auch sichtbar.

Für *inkludierte Dateien*, die nur prozeduralen Code (Funktionen, Variablen, Konstanten) enthalten, kommt oft eine Namenskonvention mit der Dateiendung `.inc.php` (z. B. `functions.inc.php`) zur Anwendung. Dateien, die eine Klasse enthalten, werden in der Regel mit dem Namen der Klasse in PascalCase und der regulären PHP-Endung benannt (z. B. `MyClass.php`). Es kann aber jede Datei mit jeder beliebigen Endung eingebunden werden.

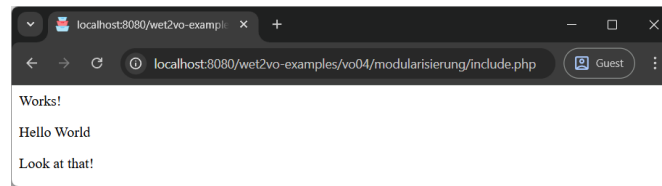


Abbildung 4.1: Obwohl `include.php` aufgerufen wurde, werden die Inhalte der anderen beiden Dateien angezeigt.

include

Mit dem Sprachkonstrukt `include "filename"`¹ wird der Inhalt einer externen Datei an der Stelle des Aufrufs inkludiert, d. h. eingefügt. Als Argument kann entweder ein relativer Pfad (z. B. `include "functions.inc.php"`) oder ebenso ein absoluter (z. B. `include "/var/www/html/functions.inc.php"`) angegeben werden. Es empfiehlt sich jedoch immer, den relativen Pfad zu verwenden. Dieser bezieht sich immer auf die aktuelle Datei. Wird dort nichts gefunden, werden die Verzeichnisse, die bei der Direktive `include_path` in der Datei `php.ini` angegeben sind, durchsucht. Wird dort auch nichts gefunden, wird eine Warnung ausgegeben, das aktuelle PHP-Skript wird jedoch weiter abgearbeitet.

Ein Beispiel. Gegeben sei die Include-Datei `vars.inc.php` mit folgendem Inhalt:

```
1 <?php
2 $test = "<p>Works!</p>";
3 const MESSAGE = "<p>Hello World</p>";
```

Diese soll in der Datei `include.php` eingebunden und ihr Inhalt verwendet werden. Dies ist ganz einfach möglich:

```
1 <?php
2 include "vars.inc.php";
3 echo $test;
4 echo MESSAGE;
```

Der Inhalt der Variablen und Konstanten wird genauso übernommen, als ob sie direkt in der Datei stünden.

Steht eine `return`-Anweisung am Ende einer inkludierten Datei, kann dieser Wert auch direkt weiterverwendet werden. Gegeben sei der Inhalt der Datei `return.inc.php`:

```
1 <?php
2 return "<p>Look at that!</p>";
```

Der dort zurück gegebene Wert kann wie folgt z. B. in `include.php` ausgegeben werden:

```
1 echo include "return.inc.php"; // <p>Look at that!</p>
```

Abbildung 4.1 zeigt den Aufruf von `include.php` im Browser.

¹<https://www.php.net/manual/de/function.include.php>

require

`require`² funktioniert gleich wie `include`. Wird jedoch die einzubindende Datei nicht gefunden, resultiert `require` in einem Fehler (anstatt einer Warnung) und das Skript wird abgebrochen.

include_once

Das Sprachkonstrukt `include_once`³ funktioniert auch wie `include`, stellt jedoch sicher, dass einzubindende Files nur ein Mal inkludiert werden. Dies ist dann sinnvoll, wenn mehrere Dateien ein und dieselbe Include-Datei einbinden und ein mehrfaches Inkludieren problematisch sein könnte.

require_once()

`require_once`⁴ ist wiederum das Pendant zu `include_once`. Der Unterschied ist hier derselbe wie zwischen `include` und `require`: Wird die einzubindende Datei nicht gefunden, bricht `require_once` mit einem Fehler ab.

4.2 Objektorientierung in PHP

Seit Version 5.0 unterstützt PHP ein ausgedehntes Objektmodell. Objektorientierung selbst war bereits in PHP 4 möglich, jedoch noch sehr eingeschränkt. Das Objektmodell ist dabei sehr stark an das von Java angelehnt.

4.2.1 Allgemeines und Terminologie

Objektorientierung ist ein fundamentales Prinzip in der Programmierung. Es impliziert eine Verbindung zwischen Daten und dem Code, der diese Daten verarbeitet. In einer rein funktionalen Sprache (wie PHP in dieser Vorlesung bis jetzt verwendet wurde) existieren die Daten lediglich in Datenstrukturen (z.B. Arrays). Dazu gibt es Funktionen, die mit diesen Daten arbeiten. In einer objektorientierten Sprache existieren Objekte. Diese speichern einerseits die Daten und beinhalten gleichzeitig Methoden, die diese Daten verarbeiten können. Dieses Konzept ermöglicht ein besseres Softwaredesign, leichtere Wartbarkeit und eine bessere Wiederverwendbarkeit des Codes. Zunächst sollen überblicksartig die wichtigsten Begriffe kurz erläutert werden:

Klasse

Als *Klasse* werden im objektorientierten Modell die Baupläne für Objekte bezeichnet. Eine Klasse definiert, welche Daten gespeichert werden (*Eigenschaften*) und welche Operationen auf diese Daten durchgeführt werden können (*Methoden*).

²<https://www.php.net/manual/de/function.require.php>

³<https://www.php.net/manual/de/function.include-once.php>

⁴<https://www.php.net/manual/de/function.require-once.php>

Objekt

Ein *Objekt* ist eine konkrete Instanz einer Klasse. Von einer Klasse kann es beliebig viele Objekte geben. Während die Klasse vorgibt, welche Daten gespeichert werden können, enthält jedes Objekt sein eigenes Set an solchen Daten.

Die Daten selbst werden *Eigenschaften* (engl. „properties“) oder auch Member-, Instanz- oder Exemplarvariablen genannt.

Die Funktionen, die den Zugriff auf die Daten des Objekts ermöglichen, werden im objektorientierten Kontext *Methoden* genannt.

Interface

Als *Interface* wird ein Set an vorgegebenen Methoden und Eigenschaften genannt, an das sich eine Klasse halten kann. Dies erleichtert die Kommunikation zwischen einzelnen Klassen, da eine zweite Klasse sich darauf verlassen kann, dass bestimmte Methoden vorhanden sind, wenn ein Interface implementiert ist.

Vererbung und abstrakte Klasse

Das Konzept der *Vererbung* ermöglicht es, dass Klassen von anderen abgeleitet werden. Sie übernehmen dabei die Eigenschaften und Methoden der sogenannten Eltern- oder Super-Klasse und können neue hinzu definieren. Soll verhindert werden, dass die Eltern-Klasse nicht instanziiert wird, kann diese als *abstrakte Klasse* definiert werden. Die abgeleitete Klasse bleibt davon unbetroffen.

Trait

Als *Trait* wird eine wiederverwendbare Sammlung an Eigenschaften und Methoden bezeichnet, die jedoch keine vollständige Klassen ist. Traits werden in mehreren unterschiedlichen Klassen eingebunden, wenn diese dieselben Funktionalitäten benötigen, eine Vererbung aber aufgrund einer zu großen Verschiedenheit in der Natur der Klassen nicht sinnvoll ist.

Enums

Enums, im Deutschen auch Aufzählungen genannt, sind spezielle Arten von Objekten. Mit ihnen wird ein eigener Datentyp spezifiziert, der nur eine bestimmte Menge an Werten annehmen kann.

4.2.2 Objekte erzeugen

PHP-Objekte werden – wie in anderen Sprachen auch üblich – mit dem Schlüsselwort `new` erzeugt. Die Syntax lautet:

```
1 $object = new Class();
```

Die Klammern sind dabei optional, sodass auch `$object = new Class;` geschrieben werden könnte.

Manche Klassen erlauben es, beim Anlegen eines Objekts Argumente mitzugeben. Diese werden in Klammern angegeben:

```
1 $object = new Class($arg1, $arg2);
```

Das folgende Beispiel legt zwei Objekte der Klasse `Rectangle` an. Einmal ohne Parameter, einmal mit fünf Parametern:

```
1 $rect1 = new Rectangle();  
2 $rect2 = new Rectangle(45, 60, 110, 120, $green);
```

4.2.3 Zugriff auf Objekteigenschaften und -methoden

Sobald ein Objekt angelegt wurde, kann auf seine Eigenschaften und Methoden zugegriffen werden. Für den Zugriff wird der Objekt-Operator `->` verwendet:

```
1 $object->property;  
2 $object->method();
```

Beim Zugriff ist darauf zu achten, dass das `$`-Zeichen nur vor die Objektvariable gesetzt wird und nicht mehr vor die Member-Variable. Ebenfalls zu beachten ist der Zugriffsmodifikator. Nur auf Eigenschaften und Methoden, die `public` sind, kann auf diese Art zugegriffen werden (siehe dazu auch Abschnitt 4.2.4).

Im folgenden Beispiel werden die Eigenschaften eines der zuvor angelegten Rechtecke geändert und eine Methode aufgerufen:

```
1 $rect1->x1 = 5;  
2 $rect1->y1 = 5;  
3 $rect1->x2 = 10;  
4 $rect1->y2 = 10;  
5 $rect1->color = $red;  
6 $rect1->draw();
```

4.2.4 Klassen deklarieren

Klassen stellen die Baupläne von Objekten dar. Sie definieren, welche Daten gespeichert werden und welche Methoden vorhanden sind. Um eine Klasse zu definieren, wird das Schlüsselwort `class` benötigt. Danach folgt der Name der Klasse. Bei diesem spielt Groß- und Kleinschreibung keine Rolle, üblicherweise werden Klassen jedoch im *PascalCase* (auch *StudlyCaps* genannt), d. h. mit großem Anfangsbuchstaben und Großbuchstaben für weitere Wörter, verfasst.

Eine Klasse wird üblicherweise in einer eigenen Datei gespeichert (die auch den Namen der Klasse trägt), es ist aber auch genauso möglich, mehrere Klassen in einer Datei zu definieren.

Nach dem Namen der Klasse folgen geschwungene Klammern, innerhalb derer alle Eigenschaften und Methoden angegeben werden. Die Syntax dazu sieht wie folgt aus:

```
1 class ClassName  
2 {  
3     private $property;  
4  
5     public function method()  
6     {  
7         // ...  
8     }  
9 }
```

Eigenschaften

Eigenschaften oder Exemplarvariablen einer Klasse geben an, welche Daten später von einem Objekt gespeichert werden können. Sie sind optional, d. h. eine Klasse muss nicht zwingend Eigenschaften aufweisen, in der Regel sind aber solche vorhanden. Eigenschaften werden meist am Beginn einer Klasse deklariert und in *camelCase* geschrieben.

Zugriffsmodifikatoren: PHP ermöglicht es, analog zu Java und anderen Sprachen, die Sichtbarkeit von deklarierten Eigenschaften anzugeben. Dafür stehen drei (bekannte) Schlüsselwörter zur Verfügung:

- **public:** Die Eigenschaft kann von überall her (außerhalb des Objekts, von anderen Klassen etc.) verwendet werden.
- **protected:** Auf die Eigenschaft kann nur von Objekten der eigenen Klasse und ggf. davon abgeleiteten Klassen verwendet werden.
- **private:** Die Eigenschaft kann nur von Methoden der eigenen Klasse verändert werden.

Die Standardsichtbarkeit ist **public**. Generell sollte der Zugriffsschutz immer sehr streng gewählt werden, um das Prinzip der Datenkapselung aufrecht zu erhalten. Der Zugriff kann dann geordnet über Methoden oder Property Hooks (s.u.) erfolgen.

Das folgende Beispiel deklariert die Eigenschaften der Klasse **Rectangle**. Dies sind zwei (x/y) -Koordinatenpaare für die Punkte links oben und rechts unten und eine Farbe. Um den Zugriff wie in Abschnitt 4.2.3 zu gewährleisten, müssen in diesem Fall alle Eigenschaften als **public** deklariert werden:

```
1 class Rectangle
2 {
3     public $x1;
4     public $y1;
5     public $x2;
6     public $y2;
7     public $color;
8
9     // ...
10 }
```

Typed Properties: Wie auch bei Funktionen können Typdeklarationen bei Eigenschaften einer Klasse verwendet werden. Diese werden zwischen dem Zugriffsmodifikator und dem Namen der Eigenschaft angegeben. Um für die Klasse **Rectangle** nur Ganzzahlen als Koordinaten sowie einen String für die Farbe zuzulassen, sieht dies wie folgt aus:

```
1 class Rectangle
2 {
3     public int $x1;
4     public int $y1;
5     public int $x2;
6     public int $y2;
7     public string $color;
8
9     // ...
10 }
```

Wie bei Funktionen (Methoden) und Rückgabewerten erzeugt die Typisierung hier einen Fehler, wenn andere Datentypen als die angegebenen zugewiesen werden.

Asymmetrische Sichtbarkeit: Seit PHP 8.4 ist es möglich, die Zugriffsmodifikatoren asymmetrisch zu gestalten. Dies ist vor allem dann sinnvoll, wenn der setzende/schreibende Kontext strenger als der lesende sein soll. So könnte etwa für die Eigenschaft `$color` festgelegt werden, dass diese zwar grundsätzlich `public` ist, der schreibende Zugriff aber `private` sein soll. Dies bedeutet, dass die Eigenschaft zwar öffentlich gelesen, aber nur intern in der Klasse gesetzt/geschrieben werden kann. Dazu wird eine zweite Sichtbarkeit mit dem Modifikator `set` für schreibend (oder auch `get`) für lesend hinzugefügt. In der Klasse `Rectangle` sieht dies dann so aus.

```
1 class Rectangle
2 {
3     public int $x1;
4     public int $y1;
5     public int $x2;
6     public int $y2;
7     public private(set) string $color;
8
9     // ...
10 }
```

Die `set`-Sichtbarkeit muss dabei gleich oder restriktiver als die Standardsichtbarkeit (das erste Schlüsselwort) sein, außerdem funktioniert asymmetrische Sichtbarkeit nur mit typed Properties. Ist die Standardsichtbarkeit `public`, so kann diese auch weggelassen werden. In diesem Fall wäre also `private(set) string $color` ausreichend.

Methoden

Eine *Methode* ist eine Funktion innerhalb einer Klasse. Sie wird genau gleich mit dem Schlüsselwort `function` definiert, jedoch wird in der Regel auch ein Zugriffsmodifikator (`public`, `protected` oder `private`) vorgestellt, um bestimmen zu können, wann die Methoden aufgerufen werden darf.

Zugriff auf Eigenschaften und Methoden: Um auf die Eigenschaften und Methoden des aktuellen Objekts zugreifen zu können, muss das Schlüsselwort `$this` benutzt werden. Es referenziert das aktuelle Objekt. Um also auf die Eigenschaft `$x1` eines `Rectangle` Objekts in einer Methode des Objekts zugreifen zu können, muss dieser Zugriff immer über `$this->x1` erfolgen. Würde nur `$x1` verwendet, so wird eine lokale Variable angelegt bzw. verwendet werden.

Im folgenden Beispiel wird eine Methode `getX1()` implementiert, die den x -Wert des linken oberen Punktes im Rechteck zurückgibt. Die Methode wird als `public` deklariert, um den Zugriff von außerhalb zu ermöglichen. Weiters wird eine Funktion `move(int $dx, int $dy)` angelegt, die alle Koordinaten um die Werte `$dx` und `$dy` verschiebt. Diese beiden Werte sind die (als `int` typisierten) Parameter der Methode. Schließlich soll es noch eine Funktion `draw()` geben, die das Rechteck zeichnet. Ihr Inhalt würde Funktionen einer Grafikbibliothek aufrufen, um das Rechteck darstellen zu können.


```

1 class Rectangle
2 {
3     public int $x1;
4     public int $y1;
5     public int $x2;
6     public int $y2;
7     public private(set) string $color;
8
9     public function move(int $dx, int $dy): void
10    {
11        $this->x1 += $dx;
12        $this->y1 += $dy;
13        $this->x2 += $dx;
14        $this->y2 += $dy;
15    }
16
17    public function getX1(): int
18    {
19        return $this->x1;
20    }
21
22    public function draw(): void
23    {
24        // Zeichnen des Rechtecks
25    }
26 }

```

Property Hooks: In PHP 8.4 wurden so genannte Property Hooks eingeführt, welche Getter- und Setter-Methoden (wie `getX1()`) überflüssig machen. Dabei wird die Definition der Eigenschaft nicht mit einem Strichpunkt sondern mit geschwungenen Klammern (`{ }`) beendet. In den Klammern finden sich die Schlüsselwörter `get` oder `set` die wiederum in geschwungenen Klammern den Getter- bzw. Setter-Code beinhalten. Für die Eigenschaft `$x1` sieht dies in der Klasse wie folgt aus:

```

1 class Rectangle
2 {
3     public int $x1 {
4         get {
5             return $this->x1;
6         }
7     }
8
9     // ...
10 }

```

Der Zugriff erfolgt dann direkt über die Eigenschaft: `$rect1->x1` anstatt über `$rect1->getX1()`. Hierbei verbindet sich die Einfachheit des Zugriffs auf eine **public** Eigenschaft mit der Sicherheit einer Getter-Methode, denn letztere stellt sicher, dass ggf. noch Code ausgeführt wird, bevor der Wert zurückgegeben wird, was mit einem direkten Zugriff nicht möglich ist. Geschieht jedoch keine weitere Logik, als den Wert der Eigenschaft zurückzugeben, ist **public**-Sichtbarkeit völlig ausreichend.

Konstruktor

Beim Erzeugen eines Objekts mit `new` wird dessen *Konstruktor* aufgerufen. Mit diesem können alle wichtigen Initialisierungen vorgenommen werden. In PHP ist der Konstruktor jedoch etwas anders definiert, als es etwa in Java der Fall ist. Es handelt sich um eine sogenannte *magische Methode* (engl. „magic method“). Derlei Methoden beginnen immer mit zwei Unterstrichen (`__`), darum sollten keine eigenen Methodennamen benützt werden, die so beginnen, da sie sonst von PHP als magisch angesehen werden.

Die magische Methode für den Konstruktor lautet `__construct()`. Die Anzahl der Parameter kann dabei beliebig gewählt werden.

Im Konstruktor der Klasse `Rectangle` werden alle vier Koordinaten und die Farbe mit Standardwerten initialisiert, Typdeklarationen sichern die Initialisierung zusätzlich ab:

```
1 class Rectangle
2 {
3     public int $x1 {
4         get {
5             return $this->x1;
6         }
7     }
8     public int $y1;
9     public int $x2;
10    public int $y2;
11    public private(set) string $color;
12
13    public function __construct(
14        int $x1 = 0,
15        int $y1 = 0,
16        int $x2 = 0,
17        int $y2 = 0,
18        string $color = "white"
19    ) {
20        $this->x1 = $x1;
21        $this->y1 = $y1;
22        $this->x2 = $x2;
23        $this->y2 = $y2;
24        $this->color = $color;
25    }
26
27    // ...
28 }
```

PHP erlaubt es nicht, Methoden zu überladen, d. h. zwei Methoden mit demselben Namen aber unterschiedlichen Parametern anzugeben. Diesem Umstand kann wie im Beispiel gezeigt entgegnet werden. Durch die Verwendung von Default-Parametern ist es möglich, den Konstruktor mit einer Anzahl zwischen null und allen fünf Parametern aufzurufen und dabei immer noch eine korrekte Initialisierung zu gewährleisten. In Abschnitt 4.2.2 wurde genau dies demonstriert.

Constructor Property Promotion: Beachtet man den Konstruktor der Klasse `Rectangle` genau, so fällt auf, dass die Initialisierung von Eigenschaften meist derselbe Prozess ist: Eigenschaft deklarieren, Parameter deklarieren, das Argument der Eigenschaft zuweisen.

Dabei wird jede Eigenschaft (etwa `$x1`) viermal angegeben.

Mit PHP 8 wurde versucht, diesen Umstand zu lösen. Mit Hilfe der Constructor Property Promotion wird die Initialisierung drastisch verkürzt. Dazu wird einfach der Zugriffsmodifikator der Eigenschaft (hier überall `public`) vor den jeweiligen Parameter im Konstruktor gestellt. Der Inhalt des Konstruktors sowie die Deklaration der Eigenschaften kann entfallen. Etwaige Property Hooks wie bei `$x1` werden direkt in den Parametern inkludiert. Der Konstruktor der Klasse `Rectangle` sieht dadurch wie folgt aus:

```
1 class Rectangle
2 {
3     public function __construct(
4         public int $x1 = 0 {
5             get {
6                 return $this->x1;
7             }
8         },
9         public int $y1 = 0,
10        public int $x2 = 0,
11        public int $y2 = 0,
12        public string $color = "white"
13    ) {}
14
15    // ...
16 }
```

Destruktor

Seit PHP5 gibt es auch *Destruktoren* in PHP. Diese erfüllen den gegenteiligen Zweck eines Konstruktors, werden also bei der Zerstörung eines Objekts aufgerufen. Dies geschieht, wenn die letzte Referenz auf ein Objekt entfernt wird (z. B. dadurch, dass die einzige Variable, die auf das Objekt zeigt, auf `null` gesetzt wird) oder ganz einfach das Ende des Skripts erreicht wird. PHP räumt alle seine Ressourcen automatisch auf, somit ist das Einsatzgebiet von Destruktoren sehr eingeschränkt. Sie können jedoch z. B. zum Schließen von Datenbankverbindungen oder einfach zum Debugging verwendet werden (um loggen zu können, wann ein Objekt zerstört wurde).

Ein Destruktor ist ebenfalls eine magische Methode und wird mit dem Namen `__destruct()` deklariert. Im Destruktor der Klasse `Rectangle` wird eine simple Meldung ausgegeben, wenn das Objekt zerstört wird.

```
1 class Rectangle
2 {
3     // ...
4
5     public function __destruct()
6     {
7         echo "Destructor called!";
8     }
9
10    // ...
11 }
```

Statische Elemente

Zusätzlich zu normalen Eigenschaften eines Objekts können auch *statische Elemente* definiert werden. Während Eigenschaften (Member-Variablen) für jedes Objekt einzeln existieren und für jede Instanz einen eigenständigen Wert haben, existiert eine statische Variable genau ein Mal pro *Klasse*. Ihr Wert ist also über alle Objekte der Klasse hinweg gleich.

Statische Variablen werden mit Hilfe des Schlüsselworts **static** angelegt. Auch für sie gelten dieselben Zugriffsmodifikatoren wie für Eigenschaften oder Methoden. Der Zugriff auf eine statische Variable, erfolgt jedoch immer über die Angabe der Klasse, *nicht* über das Objekt. Handelt es sich um eine statische Variable, die als **public** deklariert wurde, kann der Zugriff von außerhalb über die folgende Syntax erfolgen:

```
1 ClassName::$staticVar;
```

Soll innerhalb eines Objekts auf die statische Variable zugegriffen werden, so geschieht dies ebenfalls über eine derartige Syntax. Nur kann der Name der aktuellen Klasse durch das Schlüsselwort **self** ersetzt werden (da im Objekt ja der Name der Klasse bekannt ist). Die zwei Doppelpunkte werden Gültigkeitsbereichsoperator (engl. „scope resolution operator“), oder offiziell Paamayim Nekudotayim (aus dem Hebräischen für „zweimal Doppelpunkt“) genannt.

Im **Rectangle**-Beispiel wird eine statische Variable **\$version** angelegt, welche die Versionsnummer der Klasse enthält. Diese Nummer wird einmal außerhalb des Objekts und einmal über die Methode **getVersion()** innerhalb angefragt und nach außen zurück gegeben. In der letzten Zeile wird versucht, auf die statische Variable wie auf eine normale Eigenschaft zuzugreifen. Dies führt zu einer Notice über den falschen Zugriff auf ein statische Property und zu einem Warning, dass die Eigenschaft, auf die Zugriffen wird, nicht existiert.

```
1 class Rectangle
2 {
3     // ...
4     public static string $version = "1.0.0";
5
6     // ...
7
8     public function getVersion(): string
9     {
10         return self::$version;
11     }
12
13     // ...
14 }
15
16 $rect1 = new Rectangle();
17 echo Rectangle::$version; // 1.0.0
18 echo $rect1->getVersion(); // 1.0.0
19 echo $rect1->version; // NOTICE & WARNING!
```

Statische Variablen können auf dieselbe Art und Weise verändert werden. Über **Rectangle::\$version = "2.0.0"**; kann der Wert jederzeit neu gesetzt werden.

Ebenso können Methoden statisch deklariert werden. Dies bewirkt ebenfalls, dass sie zur Klasse und nicht zum Objekt gehören und somit ohne konkrete Instanz aufge-

rufen werden. Im Unterschied zu statischen Eigenschaften kann jedoch eine statische Methode auch über eine Instanz aufgerufen werden, wenngleich dies nicht sinnvoll ist und vermieden werden sollte.

Das folgende Beispiel definiert eine statische Methode, die die statische Variable `$version` mit `echo` ausgibt. Der Aufruf wird zunächst statisch, d. h. über den Klassennamen und dann am Objekt durchgeführt.

```
1 class Rectangle
2 {
3     // ...
4     public static string $version = "1.0.0";
5
6     // ...
7
8     public static function printVersion(): void
9     {
10         echo self::$version;
11     }
12
13     // ...
14 }
15
16 $rect1 = new Rectangle();
17 Rectangle::printVersion(); // 1.0.0
18 $rect1->printVersion(); // Funktioniert, aber nicht nötig/empfehlenswert
```

Konstanten

Genauso wie bei globalen Konstanten können *Klassen-Konstanten* mit dem Schlüsselwort `const` angelegt werden, seit PHP 8.3 ist auch eine Typdeklaration möglich. Ihr Wert kann nicht mehr geändert werden, sobald eine Konstante einmal angelegt wurde. Wie auch bei globalen Konstanten hat es sich eingebürgert, Großschreibung zu verwenden.

Konstanten können seit PHP 7.1 mit einem vorangestellten Zugriffsmodifikator wie `public`, `protected` oder `private` verwendet werden. In früheren Versionen bzw. ohne Modifikator sind sie automatisch `public`.

Für die Klasse `Rectangle` wird eine Konstante `TYPE` eingeführt, die den Typ des geometrischen Objekts („Rectangle“) zurückgibt. Der Zugriff erfolgt wiederum von außerhalb, über eine Methode und schließlich über das Objekt. Zugriff Nummer drei schlägt erneut fehl, da auch Konstanten in der Klasse und nicht beim Objekt gespeichert sind.

```
1 class Rectangle
2 {
3     // ...
4
5     public const string TYPE = "Rectangle";
6
7     // ...
8
9     public function getType(): string
10    {
11        return self::TYPE;
12    }
```

```

13
14 // ...
15 }
16
17 $rect1 = new Rectangle();
18 echo Rectangle::TYPE; // Rectangle
19 echo $rect1->getType(); // Rectangle
20 echo $rect1->TYPE; // FEHLER!

```

Schreibgeschützte Eigenschaften

Mit PHP 8.1 wurden *schreibgeschützte Eigenschaften* (engl. „readonly properties“) eingeführt. Dabei handelt es sich um Eigenschaften der Klasse, denen nur einmal ein Wert zugewiesen werden kann. Dies mag auf den ersten Blick wie eine Konstante wirken – für diese gilt prinzipiell dasselbe – Konstanten sind jedoch Klassenkonstanten, d. h. sie existieren bei der Klasse und nicht beim Objekt. Eine Konstante gilt also für alle Objekte einer Klasse und hat dort denselben Wert. Nur lesbare Eigenschaften existieren im Kontext des Objekts und können für jedes Objekt einen unterschiedlichen Wert haben.

Um eine nur lesende Eigenschaft zu definieren, wird das Schlüsselwort **readonly** zwischen Zugriffsmodifikator und Typhinweis gestellt. Letzteres ist zwingend nötig, denn nur schreibgeschützte Eigenschaften funktionieren nicht ohne die Angabe des Typs.

Im folgenden Beispiel wird eine nur lesbare Eigenschaft `$id` eingeführt. Ihr wird im Konstruktor ein Argument zugewiesen. Es ist keine Default-Wert angegeben, da dies nicht zielführend ist (man könnte ihn nicht mehr ändern) und es kommt wieder Constructor Property Promotion zum Einsatz, um den Zuweisungsprozess im Konstruktor zu verkürzen. Im Anschluss wird ein Objekt mit der verpflichtenden ID angelegt, für alle anderen Parameter werden die Standardwerte angenommen. Beim Versuch, die ID durch eine Zuweisung nochmals zu ändern, wird ein Fehler angezeigt.

```

1 class Rectangle
2 {
3     public function __construct(
4         public readonly int $id,
5         public int $x1 = 0,
6         public int $y1 = 0,
7         public int $x2 = 0,
8         public int $y2 = 0,
9         public string $color = "white"
10    ) {
11    }
12
13    // ...
14 }
15
16 $rect1 = new Rectangle(1);
17 $rect1->id = 3; // FEHLER!

```

Schreibgeschützte Klassen: In PHP 8.2. wurden zudem *readonly Klassen* eingeführt. Hätte eine Klasse nur Eigenschaften mit **readonly** Eigenschaften, so kann nun einfach die gesamte Klasse derart markiert werden. Das Schlüsselwort entfällt dann bei den einzelnen Eigenschaften.

Aus folgendem Beispiel zu einer hypothetischen `Post`-Klasse (etwa für einen Blog), bei dem die Daten über ein Posting einmal festgelegt und dann nicht mehr veränderbar sind...

```
1 class Post
2 {
3     public function __construct(
4         public readonly string $title,
5         public readonly string $author,
6         public readonly string $content
7     ) {
8     }
9 }
```

wird dann dieses:

```
1 readonly class Post
2 {
3     public function __construct(
4         public string $title,
5         public string $author,
6         public string $content
7     ) {
8     }
9 }
```

4.2.5 Interfaces

Ein *Interface* (zu deutsch „Schnittstelle“) stellt eine Art Vertrag da, den eine Klasse eingeht. Im Interface selbst sind Methodenprototypen und ggf. Property Hooks und Konstanten angegeben, die eine Klasse konkret implementieren muss, wenn sie dieses Interface einsetzt. Dies ist wertvoll, da andere Klassen sich darauf verlassen können, dass bestimmte Methoden vorhanden sind, wenn ein gewisses Interface implementiert wird. Ob eine Klasse ein Interface implementiert, lässt sich wie bei Vererbung (siehe Abschnitt 4.2.6) durch den `instanceof` Operator prüfen.

Interfaces werden ähnlich wie Klassen definiert. Als Schlüsselwort dient hierbei jedoch `interface`. Zudem werden Methoden in Interfaces nur angeführt und mit einem Strichpunkt abgeschlossen. Die konkrete Implementierung in geschweiften Klammern entfällt. Dies muss dann in der Klasse geschehen, die das Interface implementiert.

Um bei einer Klasse anzudeuten, dass sie ein Interface implementiert (also den Vorgaben daraus folgt), wird neben den Klassennamen das Schlüsselwort `implements`, gefolgt vom Namen des Interface, gestellt.

Umgelegt auf das `Rectangle`-Beispiel, wird ein neues Interface mit dem Namen `GeometricComponentInterface` erstellt, das allgemein jegliche geometrische Komponente repräsentieren soll. Das Interface gibt die Methoden `move(int $dx, int $dy)` (zum verschieben der Komponente) und `draw()` (zum Zeichnen der Komponente) vor.

```
1 interface GeometricComponentInterface
2 {
3     public function move(int $dx, int $dy): void;
4     public function draw(): void;
5 }
```

Die Klasse `Rectangle` implementiert nun dieses Interface mit seinen zwei vorgeschriebenen Methoden.

```
1 require "GeometricComponentInterface.php";
2
3 class Rectangle implements GeometricComponentInterface
4 {
5     // ...
6
7     public function move(int $dx, int $dy): void
8     {
9         $this->x1 += $dx;
10        $this->y1 += $dy;
11        $this->x2 += $dx;
12        $this->y2 += $dy;
13    }
14
15    // ...
16
17    public function draw(): void
18    {
19        // ...
20    }
21 }
```

Wichtig ist hierbei, dass die Methoden genau dieselben Parameter aufweisen. Voraussetzung ist, dass die Datei mit dem definierten Interface zunächst eingebunden wird, damit die Informationen darüber vorhanden sind.

Gibt es neben der Klasse `Rectangle` noch andere, die dieses Interface implementieren (z. B. `Circle` oder `Triangle`), so kann bei all diesen Klassen davon ausgegangen werden, dass sie `draw()` und `move(int $dx, int $dy)` implementieren. Es kann also etwa mit `instanceof` überprüft werden, ob ein Objekt dieses Interface implementiert und dann gefahrlos `draw()` aufgerufen werden. Gäbe es das Interface nicht, müsste überprüft werden, ob das Objekt entweder vom Typ `Rectangle` oder vom Typ `Circle` oder vom Typ `Triangle` ist, bevor die Methode zum Zeichnen aufgerufen wird.

Attribute

Attribute (englisch „Attributes“, seit PHP 8.0 verfügbar) sind Metadaten, die an Klassen, Methoden, Properties oder Parameter geheftet werden können. Sie verändern den Code nicht direkt, sondern liefern Zusatzinformationen, die entweder von der PHP-Engine selbst, von Frameworks oder von Werkzeugen wie statischen Analysetools und IDEs ausgewertet werden. Syntaktisch werden Attribute in eckigen Klammern mit vorangestelltem Hash-Zeichen notiert und der zu annotierenden Code-Stelle vorangestellt.

Ein besonders nützliches Attribut ist `#[Override]` (verfügbar seit PHP 8.3). Es wird einer Methode vorangestellt und teilt der PHP-Engine mit, dass diese Methode eine Methode einem Interface, einer Elternklasse (siehe Abschnitt 4.2.6), oder einem Trait (siehe Abschnitt 4.2.8) überschreibt bzw. implementiert. Existiert keine entsprechende Methode im Original, wirft PHP einen Fehler. Dies schützt insbesondere vor Tippfehlern im Methodennamen sowie vor Fehlern, die entstehen, wenn sich die ursprüngliche Methode nachträglich ändert (z. B. durch Umbenennung im Interface).

Im Beispiel der Klasse `Rectangle` aus dem vorigen Abschnitt lässt sich das Attribut auf die beiden vom Interface vorgegebenen Methoden anwenden:

```

1 class Rectangle implements GeometricComponentInterface
2 {
3     // ...
4
5     #[Override]
6     public function move(int $dx, int $dy): void
7     {
8         $this->x1 += $dx;
9         $this->y1 += $dy;
10        $this->x2 += $dx;
11        $this->y2 += $dy;
12    }
13
14    #[Override]
15    public function draw(): void
16    {
17        // ...
18    }
19 }
```

Würde beim Methodennamen ein Tippfehler (etwa `drwa()` statt `draw()`) passieren, so meldet PHP direkt einen Fehler, da keine entsprechende Methode im Interface existiert. Ohne das Attribut wäre der Fehler nur indirekt erkennbar, da PHP lediglich melden würde, dass `Rectangle` das Interface `GeometricComponentInterface` nicht vollständig implementiert. `#[Override]` kann also immer gesetzt werden, wenn eine Methode eine Vorgabe aus einem Interface, einer Elternklasse oder einem Trait erfüllt.

4.2.6 Vererbung

PHP kennt das Konzept der *Vererbung*. Eine Klasse kann von einer anderen erben, d. h. abgeleitet werden und übernimmt damit alle Eigenschaften und Methoden, die entweder als `public` oder `protected` deklariert sind. Um eine Klasse von einer anderen abzuleiten, wird nach dem Klassennamen das Schlüsselwort `extends` angestellt. Danach folgt der Name der Basisklasse, von der abgeleitet wird. Abgeleitete Klassen können neue Variablen und Methoden hinzufügen oder auch bestehende Methoden mit einer neuen Implementierung überschreiben.

Beim Zugriff wird bei einer überschriebenen Methode immer die Version aus der Kindklasse (d. h. der abgeleiteten Klasse) bevorzugt. Soll explizit bestimmt werden, dass die Methode der Elternklasse aufgerufen werden soll, so kann das über das Schlüsselwort `parent` und dem Gültigkeitsbereichsoperator (`::`) erfolgen. Um explizit die abgeleitete Klasse anzusprechen, kommt wieder das Schlüsselwort `self` zum Einsatz.

Im folgenden Beispiel soll eine Klasse `Box` angelegt werden, die eine dreidimensionale Repräsentation des normalen Rechtecks darstellt. Da eine Box aus sechs Rechtecken besteht, wird eine neue Variable für die Tiefe der Box eingeführt und dann die Methode `draw()` mehrmals aufgerufen. Beim ersten Mal wird die Methode der Elternklasse aufgerufen, da dies die Vorderseite, also ein normales Rechteck zeichnet. Auch hier wird das Attribut `#[Override]` angegeben, um zu kennzeichnen, dass hier eine Methode (in diesem Fall aus der Basisklasse `Rectangle`) überschrieben wird.

Damit der abgeleiteten Klasse jedoch die Informationen über die Elternklasse vorliegen, muss diese mittels `include` oder `require` eingebunden werden, falls sich die Klassendefinition nicht in derselben Datei befindet.

```
1 require "Rectangle.php";
2
3 class Box extends Rectangle
4 {
5     public int $depth;
6
7     #[Override]
8     public function draw(): void
9     {
10         // Draw the front (Rectangle)
11         parent::draw();
12         // Draw the other sides
13         // ...
14     }
15 }
```

Wichtig: Im Konstruktor einer abgeleiteten Klasse wird nicht automatisch der Konstruktor der Basisklasse aufgerufen. Ist dies gewünscht – und das ist in der Regel der Fall – so muss es explizit mit `parent::__construct()`; geschehen.

Typüberprüfungen durchführen

Soll überprüft werden, von welchem Typ ein bestimmtes Objekt ist, d. h. welche Klasse ihm zugrunde liegt, oder ob ein gewisses Interface implementiert ist, so kann der `instanceof`-Operator verwendet werden. Dabei ist zu beachten, dass ein Objekt einer abgeleiteten Klasse auch immer als ein Objekt der Basisklasse angesehen wird. Folgende Beispiele illustrieren dies:

```
1 class Human
2 {
3     // ...
4 }
5 class Employee extends Human
6 {
7     // ...
8 }
9 class Animal
10 {
11     // ...
12 }
13
14 $obj = new Employee();
15
16 if ($obj instanceof Employee) { // true
17     echo "Object is Employee";
18 }
19 if ($obj instanceof Human) { // true
20     echo "Object is Human";
21 }
22 if ($obj instanceof Animal) { // false
23     echo "Object is Animal";
24 }
```

Finale Klassen, Methoden und Konstanten

Mit dem Schlüsselwort **final** können ganze Klassen, einzelne Methoden oder seit PHP 8.1 auch Konstanten gekennzeichnet werden, um Vererbung zu verbieten bzw. einzuschränken. Werden eine Methode oder eine Konstante in der Elternklasse mit **final** gekennzeichnet, können sie in einer Kindklasse nicht überschrieben werden. Der Versuch führt zu einem Fehler. Wird die gesamte Klasse mit **final** markiert, so kann von ihr überhaupt nicht mehr abgeleitet werden.

Die Verwendung von **final** kann durchaus freizügig erfolgen. Wenn eine Vererbung nicht vorgesehen oder erwünscht ist, können dadurch ungewollte Probleme, die durch nachträgliche Codeänderungen in einer Klasse auftreten, verhindert werden.

```
1 final class NoInheritance
2 {
3     public function foo(): void
4     {
5         // ...
6     }
7 }
8
9 class CantOverrideEverything
10 {
11     public const int CAN_OVERRIDE = 1;
12     final public const int CANNOT_OVERRIDE = 2;
13
14     public function canOverride(): void
15     {
16         // ...
17     }
18
19     final public function cantOverride(): void
20     {
21         // ...
22     }
23 }
```

4.2.7 Abstrakte Klassen

Abstrakte Klassen ermöglichen es, eine Art unvollständiges Grundgerüst einer Klasse zu erstellen, das selbst nicht instanziiert werden kann. In diesem Grundgerüst sind sowohl die Klasse selbst als auch einzelne Methoden (mindestens eine) mit dem Schlüsselwort **abstract** gekennzeichnet. Dies verhindert einerseits die Instanziierung und zwingt andererseits eine Klasse, die von dieser abstrakten Klasse erbt, dass diese Methoden implementiert werden müssen. Diese Vorgehensweise ist vor allem dann sinnvoll, wenn es für eine Methode einer Elternklasse keine sinnvolle Standardimplementierung gibt, da diese sehr stark von dem jeweiligen Anwendungsfall abhängt. In diesem Fall übernehmen konkrete Kindklassen dann die jeweilige Implementierung, teilen sich aber durch die gemeinsame Elternklasse einen großen Teil des Codes, sodass dieser nicht mehrmals implementiert werden muss.

Somit unterscheiden sich abstrakte Klassen von Interfaces. Denn während auch abstrakte Klassen Methoden vorgeben, die implementiert werden müssen, können sie auch fertigen Code enthalten, der einfach vererbt wird.

Im folgenden Beispiel wird eine abstrakte Klasse erstellt, die eine abstrakte Methode und eine vollständig implementierte enthält. Die abstrakte Methode ist wie bei einem Interface mit einem Semikolon terminiert und enthält keinen konkreten Code. Durch das Schlüsselwort **abstract** bei der Methode wird angedeutet, dass diese in einer abgeleiteten Klasse implementiert werden muss. Weiters muss das Schlüsselwort **abstract** dann auch vor die Klassendefinition gesetzt werden.

```
1 abstract class AbstractClass
2 {
3     abstract protected function getValue(): string;
4
5     public function printOut(): void
6     {
7         echo $this->getValue();
8     }
9 }
```

Zu dieser Klasse gibt es eine konkrete Implementierung. Diese muss zunächst die Datei der abstrakten Klasse mit **require** einbinden und leitet danach von ihr ab. Die konkrete Klasse überschreibt nur die abstrakte Methode **getValue()**. **printOut()** wird nicht überschrieben und wird daher von der Basisklasse verwendet. Auch hier kann mit **#[Override]** gekennzeichnet werden, dass eine Methode der Basisklasse überschrieben (bzw. erst implementiert) wird.

```
1 require "AbstractClass.php";
2
3 class ConcreteClass extends AbstractClass
4 {
5     #[Override]
6     protected function getValue(): string
7     {
8         return "ConcreteClass";
9     }
10 }
```

4.2.8 Traits

PHP verwendet ein klassisches Vererbungsmodell, auch Einfachvererbung genannt. Somit kann jede Klasse nur von einer einzigen Klasse erben (d. h. abgeleitet werden). Andere Sprachen (wie etwa C++) erlauben auch Mehrfachvererbung, d. h. eine Klasse kann die Funktionalitäten von mehreren Elternklassen übernehmen.

Einfachvererbung funktioniert für den Großteil der Fälle, es gibt jedoch auch Situationen, wo sie an ihre Grenzen stößt. Etwa wenn zwei inhaltlich völlig unterschiedliche Klassen eine identische Funktionalität benötigen. Angenommen, es existieren die Klassen **Person** und **Car**, die inhaltlich sehr unterschiedlich sind. Beide jedoch sollen die Möglichkeit haben, ihre aktuelle Position (z. B. in GPS-Koordinaten) zu speichern, aktualisieren und auszugeben, also lokalisierbar zu sein (um etwa auf einer Karte darstellbar zu sein). Dafür gibt es nun mehrere Herangehensweisen:

1. Der Positionscode wird in beiden Klassen identisch implementiert.
2. Es wird ein Interface – z.B. **LocatableInterface** erstellt, das die Methoden zur Positionsbestimmung definiert.

3. Es wird eine abstrakte Elternklasse – z. B. **AbstractLocatable** – erstellt, die den Code beinhaltet und von der beide erben.
4. Es wird ein Trait – z. B. **LocatableTrait** – erstellt, der die entsprechenden Methoden definiert und implementiert und in beiden Klassen zur Anwendung kommt.

Ansatz 1 ist nicht empfehlenswert, da er Codeverdoppelung bedeutet und das DRY (Don't Repeat Yourself)-Prinzip verletzt. Die Wartbarkeit leidet ebenso, da bei einer Änderung immer beide Implementierungen geändert werden müssen.

Herangehensweise 2 ist minimal besser, da zumindest formal festgelegt wird, dass beide Klassen bestimmte Methoden definieren, mit denen das Positionsverhalten manipuliert werden kann. Allerdings muss die Implementierung immer noch in beiden Klassen erfolgen, was ebenfalls das DRY-Prinzip verletzt und schwer wartbar ist.

Ansatz 3 ist ebenfalls nicht zu empfehlen, da er eine Vererbungshierarchie zwischen zwei Klassen erzeugt, die inhaltlich nichts gemeinsam haben. Dies sollte bei Vererbung immer vermieden werden.

Die vierte Lösung, nämlich die Erstellung eines Traits, ist die beste. Ein Trait definiert sowohl die notwendigen Methoden und stellt auch gleich die Implementierung zur Verfügung. Dabei handelt es sich nicht um eine komplette Klasse, sondern um ein begrenztes Set an Funktionalität, das auch in mehrere Klassen integriert werden kann.

Traits definieren

Traits werden ähnlich wie Klassen definiert, es kommt lediglich das Schlüsselwort **trait** zum Einsatz. Ansonsten werden Eigenschaften und Methoden (inklusive Zugriffsmodifikatoren) genauso angegeben wie bei Klassen.

```
1 trait TraitName
2 {
3     private $property;
4
5     public function method()
6     {
7         // ...
8     }
9 }
```

Ein Trait zum Verwalten einer GPS-Position könnte wie folgt aussehen. Die beiden Eigenschaften **\$latitude** und **\$longitude** sind **public**, da beim Lesen und Schreiben keinerlei zusätzliche Logik erfolgt und werden mit **null** initialisiert.

```
1 trait LocatableTrait
2 {
3     public ?float $latitude = null;
4     public ?float $longitude = null;
5
6     public function getPosition(): array
7     {
8         return [$this->latitude, $this->longitude];
9     }
10
11     public function setPosition(array $pos): void
12     {
13         $this->latitude = $pos[0];
```

```
14     $this->longitude = $pos[1];
15 }
16 }
```

Traits verwenden

Um einen Trait in einer (oder mehreren Klassen) zu verwenden, wird das Schlüsselwort **use** mit dem Trait-Namen verwendet. Dadurch sind alle Methoden und Eigenschaften des Traits in der Klasse verwendbar. Zuvor muss die Datei des Traits mit **include** oder **require** eingebunden werden, um verfügbar zu sein.

Die Klasse **Person** würde den Trait wie folgt verwenden:

```
1 <?php
2
3 require "LocatableTrait.php";
4
5 class Person
6 {
7     use LocatableTrait;
8
9     // ...
10 }
11
12 $pers = new Person();
13 $pers->setPosition([48.368201, 14.514065]);
14 echo "<p>Latitude: {$pers->latitude}</p>";
15 echo "<p>Longitude: {$pers->longitude}</p>";
```

Analog funktioniert die Anwendung in weiteren Klassen wie etwa **Car**.

4.2.9 Enums (Aufzählungen)

Enums, auch Aufzählungen genannt, sind ein spezieller Typ von Klasse. Sie werden nicht dazu verwendet, einen komplexen Bauplan für ein Objekt zu definieren, sondern sind etwas reduzierter. Sie werden eingesetzt, um einen neuen Datentyp zu spezifizieren, der eine bestimmte Menge an konkreten Werten aufweist. Sie tragen zur Typsicherheit, Fehlerreduktion und Lesbarkeit bei, wann immer etwa nur bestimmte Zustände zugelassen werden sollen. Das bekannteste Beispiel für einen Enum ist der Datentyp **bool**, der die zwei Werte **true** und **false** haben kann.

Enums definieren

Ein Enum wird ähnlich wie eine Klasse, jedoch mit dem Schlüsselwort **enum** angelegt. Innerhalb der geschwungenen Klammern folgen mit **case** definierte Werten. Um etwa einen Datentyp **ExportFormat** anzulegen, der vorgibt, in welche Formate Daten exportiert werden können, könnte ein Enum wie folgt definiert werden:

```
1 enum ExportFormat
2 {
3     case XML;
4     case JSON;
5     case PDF;
6 }
```

In diesem Fall gäbe es drei mögliche Formate: XML, JSON und PDF. Etwas anderes wird nicht zugelassen. `ExportFormat` ist dabei ein eigener Datentyp, genauso wie es Klassen sind.

Enums verwenden

Dieser Enum kann nun verwendet werden, wenn eine Entscheidung zwischen diesen drei Exportformaten benötigt wird. Angenommen, es existiert die Klasse `ProductExporter`, welche eine Methode `export()` besitzt. Diese soll anhand eines Parameters entscheiden, in welches Format exportiert werden soll. Mit Hilfe des `ExportFormat`-Enums sieht dies wie folgt aus:

```
1 class ProductExporter
2 {
3     public function export(ExportFormat $format): void
4     {
5         match ($format) {
6             ExportFormat::XML => $this->exportToXML(),
7             ExportFormat::JSON => $this->exportToJSON(),
8             ExportFormat::PDF => $this->exportToPDF(),
9         };
10    }
11 }
```

`ExportFormat` ist der Datentyp des Parameters. Mit Hilfe einer `match`-Anweisung wird unterschieden, welchen Wert der Enum hat. Die Syntax entspricht hier der von Konstanten, es wird also der Name (`ExportFormat`), gefolgt vom Scope Resolution Operator (`::`) und einem der drei Werte des Enums (`XML`, `JSON`, `PDF`) angegeben. Andere Werte sind nicht möglich und führen zu einem Fehler. Danach wird etwa eine private Methode aufgerufen, die den tatsächlichen Export in das jeweilige Format durchführt.

Somit bieten Enums hier starke Typsicherheit. Würde in der Methode alternativ die Auswahl mit einem String als Parameter zugelassen (`public function export(string $format): void`), so müsste innerhalb des `match`-Ausdrucks zwangsläufig ein zusätzlicher Zweig für ungültige Eingaben überlegt werden — etwa für den Fall, dass weder `"XML"`, `"JSON"` noch `"PDF"` übergeben wird. Damit muss auch ein Fehlerfall behandelt werden: Wird der Export gar nicht durchgeführt? Soll eine Exception geworfen werden? Wie wird die Fehlermeldung angezeigt? Bei der Verwendung eines Enums als Parametertyp entfallen diese Entscheidungen, weil die fix definierten Werte sicherstellen, dass nur gültige Formate übergeben werden können – die Typsicherheit verhindert ungültige Eingaben bereits zur Übersetzungszeit.

Enum-Werte ausgeben und Backed Enums

Die Werte eines Enums können auch als String ausgegeben werden. Jeder Wert eines Enums besitzt die `name`-Eigenschaft, die den Namen als Zeichenkette ausgibt.

```
1 echo ExportFormat::XML->name; // XML
2 echo ExportFormat::JSON->name; // JSON
3 echo ExportFormat::PDF->name; // PDF
```

Im obigen Coden werden die Strings `„XML“`, `„JSON“` und `„PDF“` ausgegeben.

Sollen die einzelnen Fälle eines Enums mit Werten hinterlegt werden, kann ein so genannter *Backed Enum* verwendet werden. Möglich sind `string`- und `int`-Werte, der Datentyp wird nach dem Namen des Enums angegeben. Im folgenden wird ein Enum erzeugt, dessen drei Fälle mit Integer-Werten hinterlegt sind:

```
1 enum BackedExportFormat: int
2 {
3     case XML = 1;
4     case JSON = 2;
5     case PDF = 3;
6 }
```

Die Eigenschaft `name` ergibt nach wie vor den Bezeichner (z.B. „XML“), über die Eigenschaft `value` kann nun der Wert (z.B. „1“ für „XML“) abgefragt werden.

```
1 echo BackedExportFormat::XML->name; // XML
2 echo BackedExportFormat::XML->value; // 1
```

4.2.10 Exceptions

Zusammen mit dem objektorientierten Programmiermodell wurde in PHP 5 ein Modell zur *Ausnahmebehandlung* (engl. „exceptions“) eingeführt. Dieses Modell gleicht sehr stark dem von Java, ist jedoch bei weitem nicht so integral implementiert wie dort. So geben die meisten Funktionen aus PHP noch normale Fehlermeldungen aus und lediglich moderne Erweiterungen nützen das Exception-Modell.

Um es zu verwenden, steht die Klasse `Exception` zur Verfügung. Von ihr erzeugte Objekte repräsentieren allgemeine Fehler, die „geworfen“ und mittels eines speziellen Mechanismus behandelt werden können.

Um eine Exception zu werfen, wird das Schlüsselwort `throw` verwendet. Geworfen wird ein neues `Exception`-Objekt, dem als Parameter eine konkrete Fehlermeldung mitgegeben werden kann.

```
1 throw new Exception("General error!");
```

Jede auf diese Art ausgelöste Exception muss abgefangen und behandelt werden. Dafür steht der sogenannte `try/catch`-Block zur Verfügung. Die Exception wird im `try`-Teil ausgelöst, der `catch`-Teil spezifiziert, welcher Typ von Exception (im allgemeinsten Fall eine Ausnahme vom Typ `Exception`) abgefangen und behandelt wird. Nach der Behandlung läuft der Code normal weiter. Folgendes Beispiel zeigt einen kompletten Ablauf mit Auslösung und Behandlung des Fehlers.

```
1 try {
2     throw new Exception("General error!");
3     echo "This output isn't shown anymore.";
4 } catch (Exception $e) {
5     echo "Exception: " . $e->getMessage();
6 }
7 echo "Continue after exception handling.";
```

Sobald der Fehler mit `throw` ausgelöst wurde, stoppt die Programmausführung an dieser Stelle. Die Ausgabe mit `echo` in der Zeile darunter wird nie ausgeführt. Anstatt dessen wird zum `catch`-Block gewechselt und die Fehlerbehandlung (in diesem Fall die reine Ausgabe der Meldung) durchgeführt. Danach läuft der Code jedoch wieder weiter sodass das finale `echo` wieder angezeigt wird.

Auf einen `try`-Block können auch mehrere `catch`-Blöcke folgen, wenn verschiedene Exceptions abgefangen werden sollen. Dabei ist zu beachten, dass die spezifischeren jedoch zuerst und die allgemeinsten zuletzt kommen sollten, damit nicht die allgemeinste Exception alle spezifischeren abfängt:

```
1 try {  
2     // ...  
3 } catch (OutOfBoundsException $e) { // spezifischer  
4     echo "Exception: " . $e->getMessage();  
5 } catch (Exception $e) { // allgemeiner  
6     echo "Exception: " . $e->getMessage();  
7 }
```

Die Variable im Catch-Block (hier `$e`) kann auch weggelassen werden, wenn sie nicht verwendet wird: `catch (Exception)`.

Eigene Exceptions

Wie auch in Java können eigene Exceptions definiert werden. Dazu muss einfach von der Klasse `Exception` abgeleitet werden.

```
1 class MyException extends Exception  
2 {  
3     // ...  
4 }
```

Es empfiehlt sich jedoch, auf die vordefinierten Exceptions der Standard-PHP-Library (SPL)⁵ zurückzugreifen, in der bereits die wichtigsten Ausnahmen vordefiniert sind.

4.2.11 Namespaces

Die Beispiele von Klassen, Interfaces oder auch Traits haben demonstriert, wie sich mit Hilfe von Objektorientierung Funktionalitäten kapseln und somit auch wiederverwenden lassen. Ein Problem besteht jedoch nach wie vor: Namensgleichheit. In den Beispielen wurden Klassen wie `Rectangle`, `Person` oder `Car` verwendet. Sollen diese anderen Entwickler*innen zur Verfügung gestellt werden, besteht durchaus die Möglichkeit, dass sich in deren Code ebenfalls Klassen befinden, die diese Namen tragen und es kommt zu einem Konflikt.

Um derlei Konflikte zu vermeiden, existieren *Namespaces* (deutsch „Namensräume“). Diese gliedern PHP-Code in virtuelle Hierarchien, vergleichbar mit Verzeichnissen in einem Dateisystem. Namespaces können dabei mehrere Ebenen haben, bei der obersten spricht man vom sogenannten Vendor-Namespace. Dies sollte ein Bezeichner sein, der global einzigartig ist und somit den eigenen Code unverwechselbar zuordnet. Darunter gibt es Subnamespaces, die den Code noch weiter gliedern.

Namespaces definieren

Eine Namespace-Deklaration steht am Beginn einer PHP-Datei und wird mit dem Schlüsselwort `namespace` eingeleitet. Danach folgt der Name des Vendor-Namespace. Weitere Subnamespaces werden mit einem Backslash (`\`) getrennt angegeben. Namespaces werden in der Regel in Pascal Case angegeben.

⁵<https://www.php.net/manual/de/spl.exceptions.php>

```
1 namespace VendorNamespace\SubNamespace\AnotherSubNamespace;
```

Angenommen, es sollen in PHP Klassen erstellt werden, die Web Technologies repräsentieren. Dazu könnte es die Klassen **Lecture** und **Exercise** geben. Diese gehören inhaltlich eindeutig zusammen, somit macht es Sinn, sie unter einem gemeinsamen Namespace zu führen. Als Vendor-Namespace könnte hier z. B. **Fhooe** verwendet werden, um zu kennzeichnen, dass es sich um eine Lehrveranstaltung an der FH Oberösterreich handelt. Als Subnamespaces bieten sich dann **Mtd** und weiters **WebTechnologies** an. Somit wird die Hierarchie klar und deutlich repräsentiert.

Die Grundgerüste der beiden Klassen, zunächst die Klasse **Lecture** in **Lecture.php**:

```
1 <?php
2
3 namespace Fhooe\Mtd\WebTechnologies;
4
5 class Lecture
6 {
7     // ...
8 }
```

Danach die Klasse **Exercise** in **Exercise.php**:

```
1 <?php
2
3 namespace Fhooe\Mtd\WebTechnologies;
4
5 class Exercise
6 {
7     // ...
8 }
```

Namespaces beim Instanzieren angeben

Sollen nun in einer Datei **webtechnologies.php** Objekte von den Klassen **Lecture** und **Exercise** angelegt (d. h. instanziiert) werden, muss der Namespace mit angegeben werden. Es wird zunächst ein Objekt der Klasse **Lecture** unter Angabe des vollen Namespaces angelegt (die Klasse muss zuvor mit **include** oder **require** eingebunden werden):

```
1 $lecture = new \Fhooe\Mtd\WebTechnologies\Lecture();
```

Die Angabe des Namespaces beginnt bereits mit einem Backslash, der sozusagen den globalen Namespace darstellt, von dem ausgegangen wird. Danach folgen Vendor-Namespace, die beiden Subnamespaces und schließlich wie gewohnt der Name der Klasse. Dies wird auch als der voll qualifizierte Name einer Klasse (engl. „full qualified class name“) bezeichnet.

Da diese Angabe vor allem wenn sie öfters benötigt wird, viel Schreibarbeit bedeutet und unübersichtlich ist, können Namespaces auch für die Datei importiert werden. Dies geschieht mit dem **use** Schlüsselwort. Danach kann die Angabe des voll qualifizierten Namespaces entfallen – der Klassenname reicht. Dies soll am Beispiel eines **Exercise**-Objekts demonstriert werden:

```
1 use Fhooe\Mtd\WebTechnologies\Exercise;
2
```

```
3 // ...  
4  
5 $exercise = new Exercise();
```

Zunächst wird am Beginn der Datei der Namespace importiert, danach reicht für das Anlegen des Objekts der Klassenname.

Wird innerhalb einer Klasse, die in einem Namespace liegt, ein Objekt einer anderen Klasse instanziiert, so muss nicht der gesamte Namespace angegeben werden. PHP nimmt immer zunächst an, dass sie die Klasse im selben Namespace befindet. Würde also innerhalb der **Lecture**-Klasse ein **Exercise**-Objekt angelegt, so reicht die folgende Angabe, es muss nicht der Namespace mittels **use** importiert werden:

```
1 $exercise = new Exercise();
```

Wird ein Objekt einer Klasse angelegt, die in einem anderen Namespace liegt, muss dieses entweder als voll qualifizierter Name angegeben oder mit **use** importiert werden.

Code, der in keinem Namespace liegt, gehört zum sogenannten *globalen Namespace*. Ein gutes Beispiel hierfür ist die Klasse **Exception** (siehe Abschnitt 4.2.10). Um ein **Exception**-Objekt innerhalb einer Klasse mit Namespace anzulegen, muss der Backslash vorangestellt werden, um Explizit den globalen Namespace anzugeben. Wird darauf verzichtet oder vergessen, sucht der PHP-Interpreter im aktuellen Namespace nach einer Klasse **Exception**, wird (vermutlich) nicht fündig und erzeugt einen Fehler. Die korrekte Angabe lautet also wie folgt:

```
1 $exception = new \Exception();
```

Namespaces und Verzeichnisstruktur

PHP-Klassen, die einem Namespace angehören, können in einem beliebigen Verzeichnis liegen, da es sich bei Namespaces prinzipiell um eine virtuelle Hierarchie handelt. D. h., es müssen sich etwa die beiden Klassen **Lecture** und **Exercise** nicht zwingend in einem Verzeichnis **Fhooe>Mtd>WebTechnologies** befinden. Betrachtet man allerdings moderne PHP-Projekte, so zeigt sich schnell, dass die Hierarchie der Namespaces in der Regel auch in der Verzeichnisstruktur abgebildet wird. Der Grund hierfür ist die Autoloading-Funktionalität, die sehr stark auf diesem Prinzip basiert. Abschnitt 4.3 geht initial darauf ein, Vorlesung 5 wird die zugrundeliegenden Standards im Detail behandeln.

4.3 Autoloading

Bei der Arbeit mit Klassen und Objekten wurde ersichtlich, dass diese zunächst bekannt sein müssen, bevor sie initialisiert werden können. Sind die Klassen in eigenen Dateien definiert (was wie bereits erwähnt so üblich ist), so müssen diese Dateien zunächst mit **include** oder **require** wie in Abschnitt 4.1 beschrieben eingebunden werden. Dies funktioniert problemlos für kleine PHP-Projekte, wird jedoch umständlich, wenn nicht gar zum Problem, wenn Projekte aus mehr als nur ein paar PHP-Dateien (sprich hunderten von Dateien) bestehen.

Eine elegante und zeitgemäße Lösung stellt *Autoloading* dar. Diese Strategie bedient sich der PHP-Funktion `spl_autoload_register($autoloader)`⁶ die es erlaubt, dem

⁶<https://www.php.net/manual/de/function.spl-autoload-register.php>

PHP-Interpreter Funktionen (die sogenannten Autoloader) bekannt zu geben, die sich um das Laden von Klassen kümmern, wenn ein Objekt derselben erzeugt wird. Die Autoloader Funktion hat die Aufgabe, den voll qualifizierten Namen der Klasse, also Namespace und Klassenname, zu verwenden, um im Dateisystem die entsprechende Datei zu finden und mittels `require` einzubinden.

Um diesen Prozess einfacher zu gestalten, empfiehlt es sich, wie bereits in Abschnitt 4.2.11 erwähnt, Namespaces als Verzeichnisse abzubilden, somit muss die Autoloader-Funktion den Backslash, der die einzelnen Ebenen des Namespaces trennt, nur noch in den betriebssystemabhängigen Verzeichnisseparator umwandeln (Backslash unter Windows, Slash unter macOS bzw. Linux) und die Endung `.php` anfügen und kann dann die Datei einbinden.

4.3.1 Autoloading an einem Beispiel erklärt

Gegeben seien wiederum die Klassen `Lecture` und `Exercise`, die beide im Namespace `Fhooe\Mtd\WebTechnologies` definiert sind. Beide Dateien liegen – ausgehend vom Webverzeichnis (also dem Ordner, in dem die im Webserver aufzurufenden Dateien liegen) – unter `Fhooe\Mtd\WebTechnologies`. Im Webverzeichnis selbst liegt die Datei `autoload.php`. Sie soll von jeder der beiden Klassen ein Objekt anlegen und wird auch den Code für den Autoloader enthalten. Dieser wird wie folgt definiert:

```
1 <?php
2 function autoload(string $fullyQualifiedClassName): void
3 {
4     $fullyQualifiedClassName = ltrim($fullyQualifiedClassName, "\\");
5     $filePath = "";
6
7     if ($lastBackslashPosition = strrpos($fullyQualifiedClassName, "\\")) {
8         $namespace = substr($fullyQualifiedClassName, 0, $lastBackslashPosition);
9         $shortClassName = substr($fullyQualifiedClassName, $lastBackslashPosition + 1);
10        $filePath = str_replace("\\", DIRECTORY_SEPARATOR, $namespace) .
            DIRECTORY_SEPARATOR;
11    } else {
12        $shortClassName = $fullyQualifiedClassName;
13    }
14    $filePath .= "$shortClassName.php";
15
16    if (file_exists($filePath)) {
17        require $filePath;
18    }
19 }
20 spl_autoload_register(autoload(...));
```

In Zeile 20 wird `spl_autoload_register()` aufgerufen. Der Parameter ist eine Referenz auf eine Callback-Funktion in der First-Class Callable Syntax, in diesem Fall `autoload()`. Diese Referenz wird dem PHP-Interpreter übergeben. Somit weiß dieser, dass er die Funktion aufrufen kann bzw. muss, wenn Objekte instanziiert werden sollen, deren Klassendefinitionen nicht bekannt sind.

`autoload()` selbst ist ab Zeile 2 definiert und bekommt als Argument automatisch den voll qualifizierten Klassennamen übergeben (egal, ob dieser beim `new`-Aufruf komplett angegeben oder über `use` importiert wird). Im Falle von `Lecture` nimmt `$fullyQualifiedClassName` also den Wert `Fhooe\Mtd\WebTechnologies\Lecture` an.

Der Backslash am Anfang wird nicht mit übergeben, würde aber zur Vorsicht in Zeile 4 entfernt werden.

Als nächstes wird dieser String in den Pfad zur Klassendatei (`$filePath`) und den reinen Klassennamen ohne Namespace (`$shortClassName`) aufgeteilt. Dies geschieht ab Zeile 7, wo zunächst das letzte Vorkommen des Backslashes gesucht und dann der String von Beginn bis zu diesem Zeichen in `$namespace` und der Rest in `$shortClassName` gespeichert wird. Danach werden im String des Namespaces die Backslash-Zeichen durch den im aktuellen Betriebssystem vorherrschenden Verzeichnistrenner (festgelegt in `DIRECTORY_SEPARATOR`) ersetzt. Unter Windows bleibt der Backslash, unter macOS oder Linux wird ein Slash eingesetzt. Schließlich wird der Klassenname um die Endung `.php` ergänzt und an den bisherigen Pfad angehängt. Hat die Klasse keinen Namespace, so wird im `else`-Zweig der Klassenname unverändert übernommen und `$filePath` bleibt leer, sodass die Datei direkt im Webverzeichnis erwartet wird. Bevor der so entstandene Pfad mit `require` eingebunden wird, prüft Zeile 16 mit `file_exists()`, ob die Datei tatsächlich existiert. Andernfalls würde `require` einen Fatal Error werfen und die Ausführung sofort abbrechen – mit der Existenzprüfung kann der Autoloader bei unbekannten Klassen hingegen sauber aufgeben, sodass etwa weitere registrierte Autoloader die Möglichkeit bekommen, die Klasse zu finden.

Um Autoloading zu verwenden, muss diese Datei einmal im Projekt eingebunden sein. Sie kümmert sich fortan darum, unbekannte Klassendefinitionen zu laden. Um Objekte von `Lecture` und `Exercise`, wie in Abschnitt 4.2.11 gezeigt, zu erzeugen, muss nur der Autoloader, nicht die spezifischen Klassendefinitionen eingebunden werden:

```
1 require "autoload.php";
2
3 use Foo\Bar\WebTechnologies\Exercise;
4 use Foo\Bar\WebTechnologies\Lecture;
5
6 $lecture = new Lecture();
7 $exercise = new Exercise();
```

Diese Art des Autoloadings folgt dem Grundprinzip, dass die Verzeichnisstruktur die Namespace-Hierarchie eins zu eins abbildet. Historisch wurde dieses Vorgehen im *PSR-0 Standard*⁷ festgeschrieben, der zusätzlich auch Underscores in Klassennamen als Verzeichnistrenner interpretierte – ein Erbe aus der Zeit vor der Einführung von Namespaces in PHP 5.3. PSR-0 ist mittlerweile veraltet und durch den flexibleren *PSR-4 Standard* abgelöst, der unter anderem erlaubt, einzelne Namespace-Präfixe auf konfigurierbare Basisverzeichnisse abzubilden. Der hier gezeigte Autoloader entspricht in seiner Funktionsweise einer vereinfachten Variante von PSR-4. Letzterer wird, zusammen mit weiteren PSR-Standards, in Vorlesung 5 besprochen.

4.4 Weiterführende Literatur

Das Kapitel der Objektorientierung wird in [1] gut behandelt, einen guten Überblick bieten auch wieder [2, 3]. Hier sind auch neuere Konzepte durchaus abgedeckt, wenngleich auch nicht allzu umfangreich.

⁷<https://www.php-fig.org/psr/psr-0/>

Einen Fokus auf die Objektorientierung der Sprache legt [4], darüber hinaus werden bewährte Entwicklungsmuster (Patterns) erläutert.

Quellenverzeichnis

- [1] Kevin Tatroe und Peter MacIntyre. *Programming PHP. Creating Dynamic Web Pages*. 4. Aufl. Sebastopol, USA: O'Reilly, 31. März 2020 (siehe S. 4-28).
- [2] Thomas Theis. *Einstieg in PHP 8 und MySQL*. 15. Aufl. Bonn, Deutschland: Rheinwerk Computing, 5. Apr. 2023 (siehe S. 4-28).
- [3] Christian Wenz und Tobias Hauser. *PHP 8 und MySQL. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Mai 2024 (siehe S. 4-28).
- [4] Matt Zandstra. *PHP 8 Objects, Patterns, and Practice: Volume 1. Mastering OO Enhancements and Design Patterns*. 7. Aufl. New York, USA: Apress, 2. Dez. 2024 (siehe S. 4-29).